

Flink랑 Spark가 kafka 토픽을 처리하는 방식

Flink나 Spark 같은 처리 엔진들이 카프카 데이터를 읽어오는 방식은 **'똑똑한 컨슈머 (Smart Consumer)'**가 되어 직접 카프카 브로커에게 데이터를 달라고 요청하는 방식입니다.

이 엔진들은 단순한 컨슈머를 넘어, 대규모 데이터를 병렬로 처리하기 위해 **카프카의 파티션 (Partition) 구조**를 그대로 활용합니다. 노선에 정리하기 좋게 작동 원리를 핵심 위주로 설명해 드릴게요.

Flink와 Spark의 카프카 데이터 수집 원리

1. 전용 '커넥터(Connector)' 라이브러리 활용

Flink와 Spark는 카프카와 통신하기 위한 전용 라이브러리(애플리케이션 소스)를 내장하고 있습니다.

- **Flink:** `Flink Kafka Consumer` (또는 신규 `KafkaSource`)를 사용합니다.
- **Spark:** `Spark Structured Streaming`의 `readStream` 기능을 사용합니다.

2. 파티션-워커 매핑 (병렬 처리의 핵심)

이 엔진들이 빠른 이유는 카프카의 파티션을 엔진의 작업 단위(Worker/Task)와 1:1 또는 N:1로 매핑하기 때문입니다.

- **원리:** 카프카 토픽에 파티션이 4개 있다면, Flink나 Spark도 4개의 슬롯(Slot)을 열어 각 파티션의 데이터를 동시에 빨아들입니다.
- **효과:** 데이터가 아무리 많아도 파티션과 엔진의 CPU 코어만 늘리면 처리 속도가 무한히 확장됩니다.

3. 체크포인트와 오프셋 관리 (유실 방지)

단계	동작 방식
읽기	엔진이 카프카에서 특정 **오프셋(Offset)**의 데이터를 가져옵니다.
처리	데이터를 가공(필터링, 집계 등)합니다.

저장(체크포인트)	가공이 완료되면 엔진 자체 저장소에 *** 나 카프카 100번 오프셋까지 처리 완료했음 *** 이라고 기록합니다.
복구	엔진이 장애로 꺼졌다가 켜지면, 저장된 체크포인트를 보고 카프카 101번부터 다시 읽어옵니다.

구분	Apache Flink	Apache Spark (Streaming)
처리 방식	Real-time : 데이터가 들어오는 즉시 한 건씩 처리	Micro-batch : 짧은 시간(예: 1초) 동안 데이터를 모아서 처리
지연 시간 (Latency)	매우 낮음 (밀리초 단위)	상대적으로 높음 (초 단위)
적합한 용도	실시간 알람, 실시간 사기 탐지	대규모 로그 분석, 배치형 데이터 적재

현재 **Kafka → Flink → MinIO → Spark** 라인을 설계하셨죠?

1. **Flink**: 카프카에서 데이터를 ****실시간(Real-time)****으로 읽어와서, 필요한 형식으로 변환하거나 불필요한 데이터를 걸러낸 뒤 **MinIO**에 저장하는 역할을 수행합니다.
2. **Spark**: MinIO에 차곡차곡 쌓인 대량의 데이터(파일)를 한꺼번에 읽어서 복잡한 통계 분석을 하거나 AI 모델을 학습시키는 역할을 수행하게 됩니다.

✅ 결론

Flink와 Spark는 카프카 입장에서 보면 *****아주 성능 좋고 똑똑한 컨슈머 군단*****입니다. 파티션 단위로 데이터를 쪼개서 병렬로 읽어오기 때문에 대용량 처리가 가능한 것이죠.